

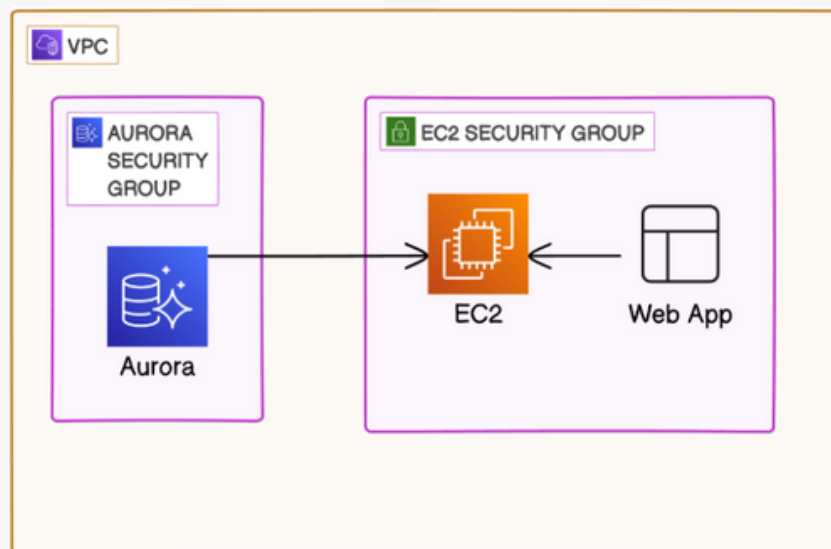


Nikko Cajayon
Community Lead



Cloud Project:

Connect Aurora Database with EC2



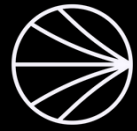


TABLE OF CONTENTS

1

INTRODUCTION

Introduction
System Architecture
Tools and services

2

LAUNCH AN EC2 INSTANCE FOR YOUR WEB APP

Understanding EC2
Create Virtual Server

3

CREATE AN AURORA MYSQL DATABASE

Understanding Aurora MySQL
creating database

4

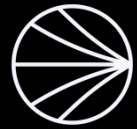
CREATE YOUR WEB APP

Configure Security group and VPC's
create Web-application

5

VIRTUAL WEB INFRASTRUCTURE

Establish the connection from local to cloud
Infrastructure



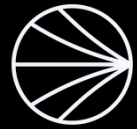
1

INTRODUCTION

In this project, I'll be recreating one of the most common and essential use cases of cloud computing: connecting a dynamic web application to a persistent database backend.

There are countless web apps on the Internet today, and engineers use databases all the time to store the web app's data, e.g., login credentials, e-commerce product information, movie reviews, user comments... you name it!

We will build a mini version of this pattern by setting up a simple web application from scratch and connecting it with the relational database (your existing Amazon Aurora cluster) to store things that users input. This final step transforms your provisioned infrastructure into a fully functional, data-driven application environment.



TOOLS AND SERVICES

Amazon Aurora

- A fast, highly available, fully managed relational database service by AWS, compatible with MySQL and PostgreSQL.

AWS Elastic Compute Cloud (EC2)

- a service offering resizable computing capacity (virtual servers) in the cloud to host applications.

Security Groups

- Acts as a firewall for controlling inbound and outbound traffic.
- Restricts database access only to approved IPs

IAM (Identity and Access Management)

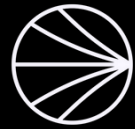
- Manages user roles and policies for AWS resources.
- Provides least-privilege access -
- Ensures only authorized users can manage or query the database.

Relational Databases

- A type of database that stores structured data in tables (rows and columns) and uses keys to define relationships between data points.

Command Line Interface (CLI)

- A text-based user interface used to issue commands directly to a computer's operating system or software programs, used here for management and configuration.

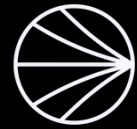


LOGIN WITH YOUR IAM USER

An IAM (Identity and Access Management) user is like a specific account you create within your AWS account. Think of it as a separate login that allows a person or application to interact with AWS services. It's a way to give access to your AWS resources without sharing your main account credentials.

Here's a more detailed breakdown of IAM users:

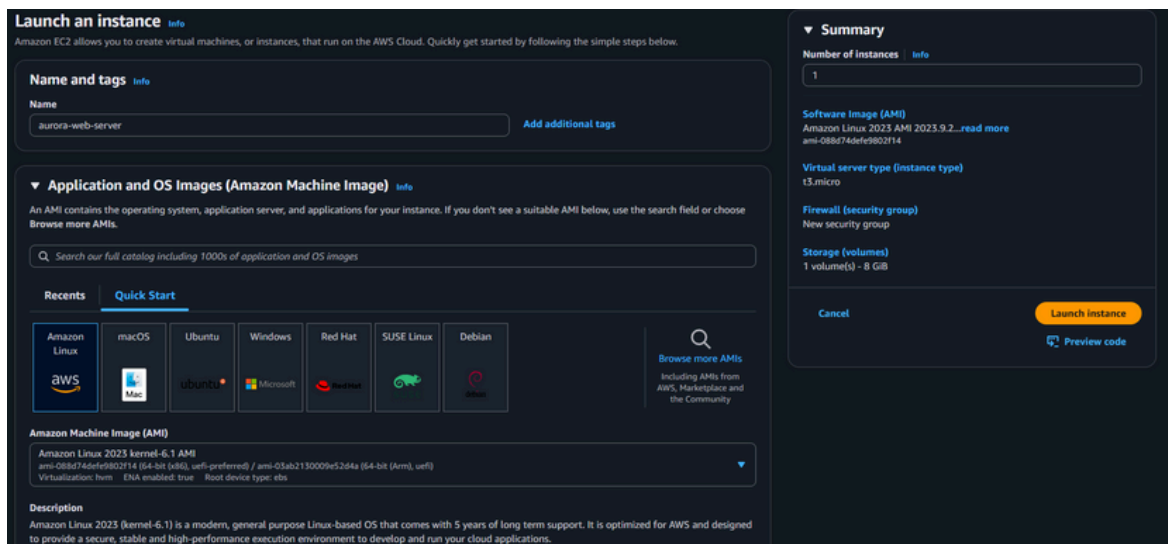
- **Security:** IAM users enhance security by allowing you to grant specific permissions. Instead of giving everyone full access, you can limit what each user can do.
- **Granular Control:** You can define exactly which AWS services and resources an IAM user can access. This is done through policies, which are like rulebooks that specify permissions.
- **Individual Credentials:** Each IAM user has their own unique username and password (or access keys for programmatic access), separate from your root account.
- **Auditing:** IAM makes it easier to track who did what in your AWS account. Since each user has their own identity, you can monitor their activities.



2

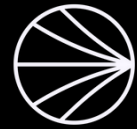
LAUNCH AN EC2 INSTANCE FOR YOUR WEB APP

This step focuses on setting up the compute environment by launching an Amazon EC2 instance. Using the AWS Management Console, we ensure the launch uses the account's default VPC and associated default subnets for simplicity.



WHAT IS AN EC2 INSTANCE

An EC2 instance is like a computer in the cloud that you can rent from Amazon Web Services (AWS). It stands for Elastic Compute Cloud, meaning you can easily change the amount of computing power you're using. Think of it as renting a virtual machine to run your applications.

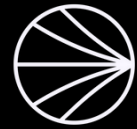


HERE'S A MORE DETAILED LOOK AT EC2 INSTANCES:

- **Virtual Servers:** meaning they simulate the hardware of a physical computer.
- **Scalability:** You can easily increase or decrease the number of instances you're using based on your needs.
- **Variety of Instance Types:** AWS offers different types of instances optimized for various workloads.
- **Operating Systems:** choose from a variety of operating systems, including Linux, Windows Server, and macOS.
- **Pay-as-you-go Pricing:** You only pay for the compute time you use, making it cost-effective for many applications.

The screenshot shows the AWS Management Console's 'Network settings' for an EC2 instance. The 'Firewall (security groups)' section is active, displaying options to 'Create security group' (selected) or 'Select existing security group'. Below, it lists rules for the new group 'launch-wizard': 'Allow SSH traffic from Anywhere' (checked), 'Allow HTTPS traffic from the internet' (unchecked), and 'Allow HTTP traffic from the internet' (unchecked). A yellow warning box at the bottom states: 'Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.'

Think about the Public IPv4 DNS as the location of a house, and then the Key pair assigned at launch like the keys that let you in. It's no good just having the location without any keys, and you can't have keys without knowing where to go! Both are critical for accessing our EC2 instan



3

CREATE AN AURORA MYSQL DATABASE

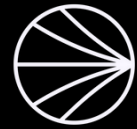
This step finalizes the database setup by controlling who can connect to it. We need to update the database's firewall (its security settings) to specifically permit traffic only from the web application's server.

WHAT IS A RELATIONAL DATABASE? HOW IS IT DIFFERENT FROM A NORMAL DATABASE?

A relational database is a type of database that organizes data into tables, which are collections of rows and columns. Kind of like a spreadsheet! We call it "relational" because the rows relate to the columns and vice-versa.

When a database is relational we can query it using a special language called SQL (Structured Query Language)

The screenshot shows the 'Engine options' section of the Amazon Aurora console. Under 'Engine type', 'Aurora (MySQL Compatible)' is selected. Below this, there are sections for 'Engine version' (set to 'Aurora MySQL 3.08.2 (compatible with MySQL 8.0.39) - default for major version 8.0') and 'Templates' (with 'Dev/Test' selected). The 'Enable RDS Extended Support' checkbox is also visible.



WHAT IS ENGINE TYPE?

in the context of databases refers to the underlying database management system (DBMS) that powers the database. Think of it as the core software that handles how data is stored, retrieved, and managed. Choosing the right engine type is crucial for performance, compatibility, and features.

AWS Aurora is a type of relational database. As you can tell from the first few steps of creating a relational database, there are plenty of options to choose from!

We'd use AWS Aurora if we needed something large-scale, with peak performance and uptime. This is because Aurora databases use clusters (more on that later!). Ordinary relational databases, like MySQL and Oracle are more generic and cost-effective. They suit smaller databases and less demanding workloads.

Instance configuration

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class | [Info](#)

▼ Hide filters

☐ Include previous generation classes

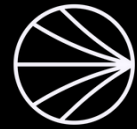
☐ Serverless v2

☐ Memory optimized classes (includes r classes)

☒ Burstable classes (includes t classes)

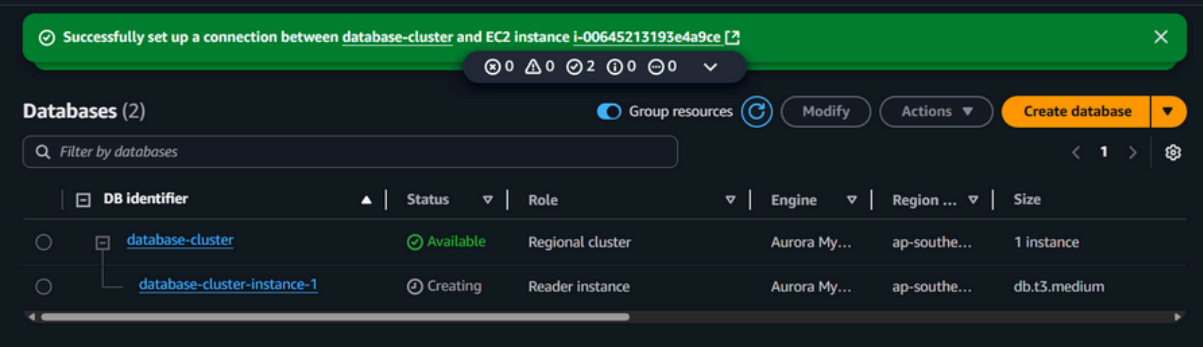
db.t3.medium

2 vCPUs 4 GiB RAM EBS Bandwidth: Up to 2,085 Mbps Network: Up to 5 Gbps



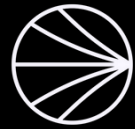
WHAT ARE BURSTABLE CLASSES?

The Burstable classes (includes t classess) are cost-effective types of database instances that are best when you have a consistent baseline level of traffic with occassional, random spikes in demand...a sudden "burst" of traffic. By choosing burstable classes, our database can perform well under high traffic, but also save us costs when things are quiet.



The screenshot shows the AWS RDS console interface. At the top, a green notification bar states: "Successfully set up a connection between database-cluster and EC2 instance i-00645213193e4a9ce". Below this, the "Databases (2)" section is visible, with a search filter "Filter by databases" and a "Group resources" toggle. A table lists the database resources:

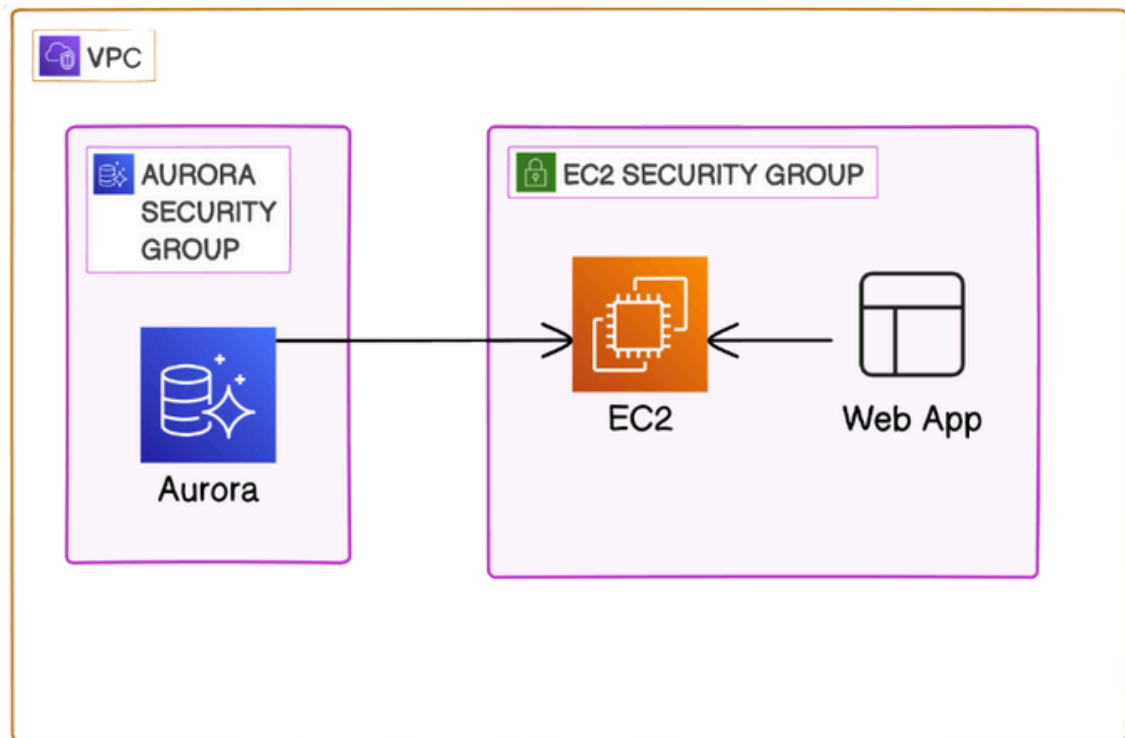
DB identifier	Status	Role	Engine	Region ...	Size
database-cluster	Available	Regional cluster	Aurora My...	ap-southe...	1 instance
database-cluster-instance-1	Creating	Reader instance	Aurora My...	ap-southe...	db.t3.medium

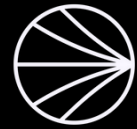


4

CREATE YOUR WEB APP

This process begins by establishing a secure SSH connection to the running EC2 instance using the key pair downloaded earlier. Once connected, the necessary web server software. and then we will set-up a secure connection between server and database.





SECURE CONNECTION

To prepare for connecting to my EC2 instance, I first need to locate my security credentials. Since the .pem file acts as the private key to my server, I searched for it on my local computer (likely in my Downloads folder) and moved it into a new folder named nextwork on my Desktop. Next, I must navigate to this specific directory using my terminal. I will run the ls command to see available directories; if I don't see Desktop, I'll use `cd ../` to move up or `cd [folder name]` to move into directories until I reach the nextwork folder on my Desktop, ensuring I am in the correct position to use the key for login.

```
Command Prompt
C:\Users\USER>cd desktop

C:\Users\USER\Desktop>dir
Volume in drive C has no label.
Volume Serial Number is AA74-8E79

Directory of C:\Users\USER\Desktop

10/02/2025  10:28 PM  <DIR>          .
09/30/2025  11:37 PM  <DIR>          ..
01/09/2025  04:24 PM  <DIR>          1-Devops
09/03/2025  02:11 PM             26,284 code_editor.xml
11/24/2024  11:58 PM  <DIR>          css-flexbox
11/25/2024  01:22 PM  <DIR>          CSS-grid
10/02/2025  10:28 PM  <DIR>          NextWork
               1 File(s)            26,284 bytes
               6 Dir(s)  229,091,655,680 bytes free

C:\Users\USER\Desktop>cd NextWork

C:\Users\USER\Desktop\NextWork>dir
Volume in drive C has no label.
Volume Serial Number is AA74-8E79

Directory of C:\Users\USER\Desktop\NextWork

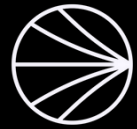
10/02/2025  10:28 PM  <DIR>          .
10/02/2025  10:28 PM  <DIR>          ..
10/02/2025  09:37 PM             1,674 NW-Aurora-App.pem
               1 File(s)            1,674 bytes
               2 Dir(s)  229,091,610,624 bytes free

C:\Users\USER\Desktop\NextWork>
```



0 means "no permission." Neither the group nor others have any permissions (cannot read, write, or execute the file).

-y means automatically answer yes to any prompts that appear during the update process.



5

VIRTUAL WEB INFRASTRUCTURE

this time I'll write sourcecode for the basic web application, likely using PHP for its simple server-side scripting to make user-friendly pages. The code will be written to use the Aurora database endpoint to securely connect the server and database

```
cd /var/www
```

```
mkdir inc
```

```
ls -ld
```

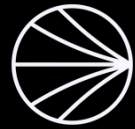
```
sudo chown ec2-user:ec2-user /var/www
```

WHAT DOES THIS DO?

The command `ls` means list, and `-ld` means "list the details".

This means that the root user is the owner of this folder, and only the owner can edit this folder. When you access an EC2 instance using a key pair, you're by default logging in as an admin user (called `ec2-user`) which is not the root user!

Let's change the owner of this folder so you (`ec2-user`) can edit it.



WHAT'S GOING ON?

Nano is a way we can edit files in the terminal. It looks a bit different, but it's nothing to be scared of!

We've just opened our blank dbinfo.inc file, and now we're going to add some code that connects it to our AWS database.

```
ec2-user@ip-172-31-13-22:/va X + v
GNU nano 8.3 SamplePage.php
</html>

<?php

/* Add an employee to the table. */
function AddEmployee($connection, $name, $address) {
    $n = mysqli_real_escape_string($connection, $name);
    $a = mysqli_real_escape_string($connection, $address);

    $query = "INSERT INTO EMPLOYEES (NAME, ADDRESS) VALUES ('$n', '$a')";

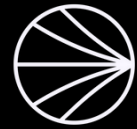
    if(!mysqli_query($connection, $query)) echo("<p>Error adding employee data.</p>");
}

/* Check whether the table exists and, if not, create it. */
function VerifyEmployeesTable($connection, $dbName) {
    if(!TableExists("EMPLOYEES", $connection, $dbName))
    {
        $query = "CREATE TABLE EMPLOYEES (
            ID int(11) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
            NAME VARCHAR(45),
            ADDRESS VARCHAR(90)
        )";
    }
}
```

```
cd /var/www/html
```

```
>SamplePage.php
```

```
nano SamplePage.php
```



Php Web-Application

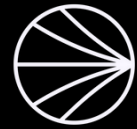
```
{ ContentCatalog.json  Release Notes: 1.105.0  <?php include "../inc/dbinfo.inc"; ?> Untitled-1
```

```
1  <?php include "../inc/dbinfo.inc"; ?>
2  <html>
3  <body>
4  <h1>Sample page</h1>
5  <?php
6
7  /* Connect to MySQL and select the database. */
8  $connection = mysqli_connect(DB_SERVER, DB_USERNAME, DB_PASSWORD);
9
10 if (mysqli_connect_errno()) echo "Failed to connect to MySQL: " . mysqli_connect_error();
11
12 $database = mysqli_select_db($connection, DB_DATABASE);
13
14 /* Ensure that the EMPLOYEES table exists. */
15 VerifyEmployeesTable($connection, DB_DATABASE);
16
17 /* If input fields are populated, add a row to the EMPLOYEES table. */
18 $employee_name = htmlentities($_POST['NAME']);
19 $employee_address = htmlentities($_POST['ADDRESS']);
20
21 if (strlen($employee_name) || strlen($employee_address)) {
22     AddEmployee($connection, $employee_name, $employee_address);
23 }
24 ?>
25
26 <!-- Input form -->
27 <form action="<?PHP echo $_SERVER['SCRIPT_NAME']; ?>" method="POST">
28     <table border="0">
29         <tr>
30             <td>NAME</td>
31             <td>ADDRESS</td>
32         </tr>
33         <tr>
34             <td>
35                 <input type="text" name="NAME" maxlength="45" size="30" />
36             </td>
37             <td>
38                 <input type="text" name="ADDRESS" maxlength="90" size="60" />
39             </td>
40         </tr>
41         <tr>
42             <td colspan="2"><input type="submit" value="Add Data" /></td>
43         </tr>
44     </table>
45 </form>
46
47
```

This big block of code is what makes our new web app look so cool! It's pulling in the details from our dbinfo.inc file we created earlier and using it to display up-to-date changes directly from our website.

Sample page

NAME	ADDRESS	
		Add Data
ID	NAME	ADDRESS



CONNECT TO YOUR DATABASE USING MYSQL CLI

MySQL CLI is a powerful, secure, and efficient tool for interacting with MySQL databases. It provides direct access to all MySQL features, supports automation, and is a preferred method for managing databases on remote servers, such as EC2 instances.

```
sudo yum install https://dev.mysql.com/get/mysql80-community-release-el7-3.noarch.rpm -y
```

```
sudo yum install mysql-community-client -y
```

```
mysql -h YOUR_WRITER_ENDPOINT -P 3306 -u admin -p
```

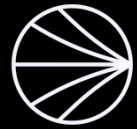
```
MySQL [sample]> SHOW TABLES;
+-----+
| Tables_in_sample |
+-----+
| EMPLOYEES        |
+-----+
1 row in set (0.003 sec)
```

```
DESCRIBE EMPLOYEES;
```

```
MySQL [sample]> DESCRIBE EMPLOYEES;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| ID    | int unsigned  | NO   | PRI | NULL    | auto_increment |
| NAME  | varchar(45)   | YES  |     | NULL    |                |
| ADDRESS | varchar(90)  | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.002 sec)
```

```
SELECT * FROM EMPLOYEES;
```

```
MySQL [sample]> SELECT * FROM EMPLOYEES;
+----+-----+-----+
| ID | NAME          | ADDRESS                               |
+----+-----+-----+
| 1  | Mr Eddy Example | 100 Example Street, New York |
| 2  | Miss Sarah     | 02 Silver Lane, San Francisco |
+----+-----+-----+
2 rows in set (0.001 sec)
```



PROJECT SUMMARY

This phase of the project successfully established the crucial, secure link between my local development environment and the cloud infrastructure.

Summary of Accomplishments:

1. **Local Key Management:** I secured my private key (.pem file) by relocating it to a designated local directory (nextwork on the Desktop), ensuring it was ready for use.
2. **Terminal Navigation:** I utilized fundamental Command Line Interface (CLI) commands (ls and cd) to efficiently navigate my local file system and position the terminal directly within the key's directory.
3. **Connection Preparation:** By successfully locating and positioning the .pem file, I completed the necessary local preparation to execute the SSH command for logging into the EC2 instance.
4. **Infrastructure Foundation:** Successfully confirming the local setup validates the initial EC2 instance launch and the integrity of the associated private key pair generated during the provisioning phase

This seemingly simple step is fundamental to all cloud development, as the .pem file is the cryptographic key that authenticates my identity to the EC2 instance. By mastering the secure handling and location of this file via the CLI, I have removed the largest hurdle to remote management. I am now fully prepared to securely SSH into the EC2 instance and proceed with the installation of the web server and application runtime in the next project phase



nextwork.org



Nikko Cajayon

NextWork Community Lead
Director for SkillBuilder
CTO of AWS Cloud Club

Here at NextWork We are
Transforming the Way World Learns

One Step Backward, Two Steps Forward